

Buffer Management in Postgres

Datenintegration und -analyse



Einführung

- 1. Aufbau des Buffercaches verstehen**
- 2. Seiten direkt in den Buffercache laden**
- 3. Seiten direkt aus dem Buffercache entfernen**

Buffer Manager

- Implementiert Paging in Postgres (src/backend/storage/buffer)
- Verwaltet welcher Block von welcher Relation im Arbeitsspeicher vorhanden ist
- Jeder Datenblock einer Relation ist als Buffer Page formatiert
- Jede Page hat einen Buffer Descriptor (Buffer Header) mit einem eigenen Spinlock
- Buffer Page identifiziert mit Buffer Tag
- Buffer Tag Lookup mit Hash Table

Clock-Sweep Seitenersetzung

- Pin über Reference Counter
(wenn > 0 kann die Seite nicht ersetzt werden)
- Aktuelle Relevanz über Usage Counter
(begrenzt mit einem limit)

NVB

	[0]	[1]	[2]	[3]	[4]	[5]
Before Clock Sweep	pin: 1 use: 3	pin: 0 use: 1	pin: 0 use: 2	pin: 0 use: 0	pin: 1 use: 1	pin: 1 use: 2

NVB

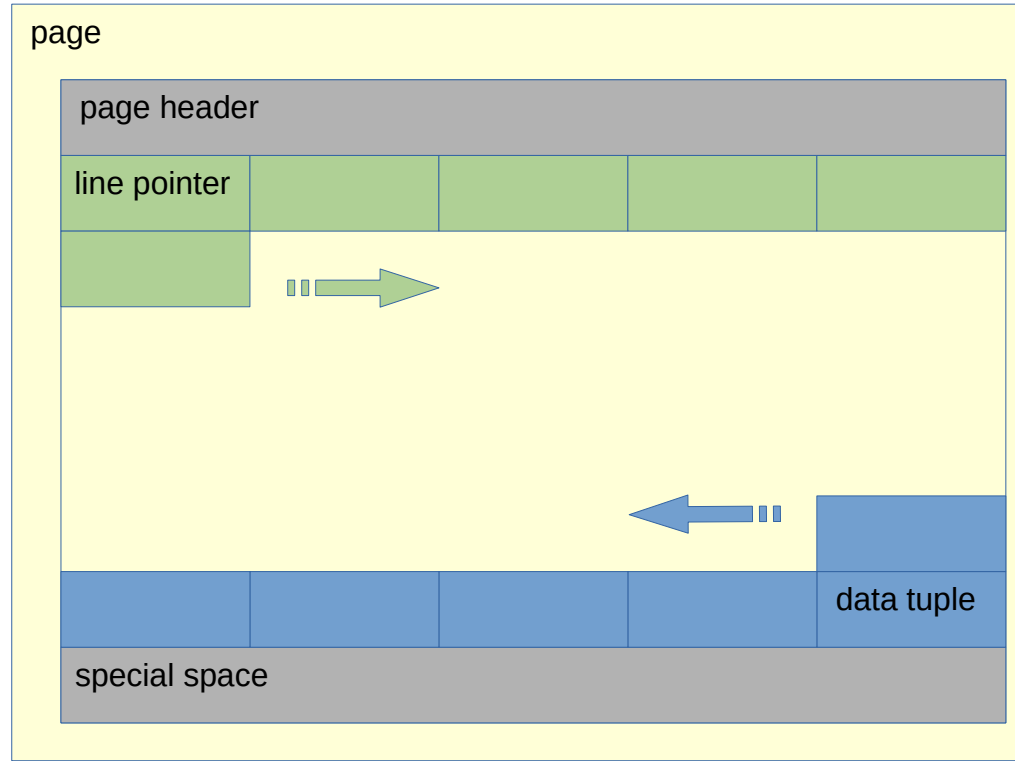
	[0]	[1]	[2]	[3]	[4]	[5]
After Clock Sweep	pin: 1 use: 2	pin: 0 use: 0	pin: 0 use: 1	pin: 1 use: 0	pin: 1 use: 1	pin: 1 use: 2

[4] Clock-sweep Replacement Strategy

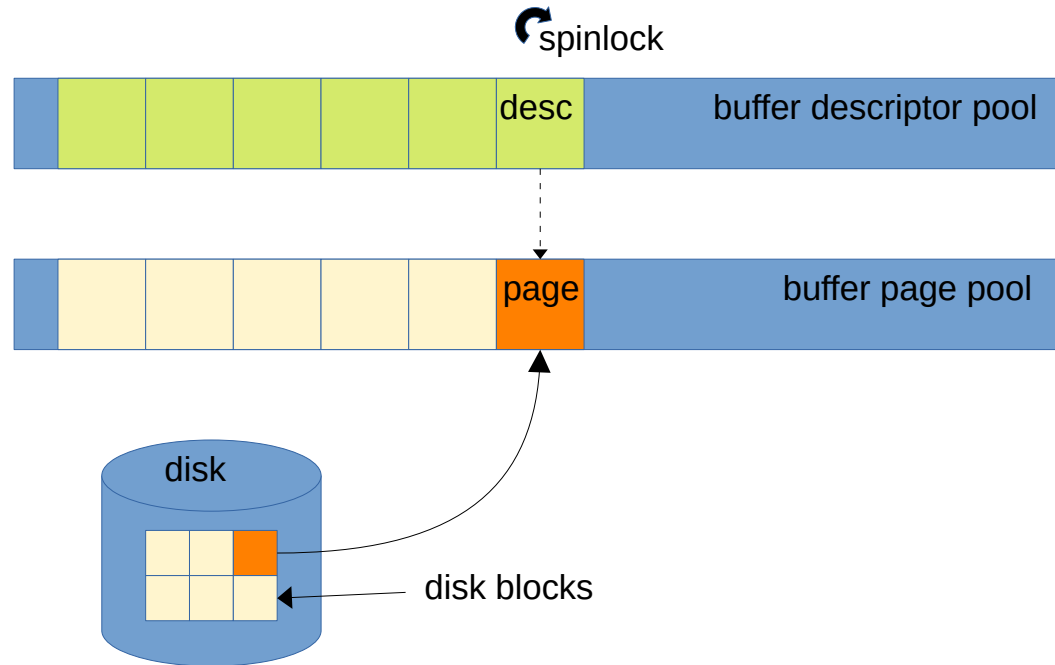


Implementierung

Buffer Page



Buffer Descriptor



buf_internals.h

```
typedef struct buftag
{
    Oid          spcOid;          /* tablespace oid */
    Oid          dbOid;          /* database oid */
    RelFileNumber relNumber;     /* relation file number */
    ForkNumber   forkNum;       /* fork number */
    BlockNumber  blkNum;        /* blknum relative to begin of reln */
} BufferTag;

typedef struct BufferDesc
{
    BufferTag      tag;           /* ID of page contained in buffer */
    int           buf_id;        /* buffer's index number (from 0) */

    /* state of the tag, containing flags, refcount and usagecount */
    pg_atomic_uint32 state;

    int           wait_backend_pgprocno; /* backend of pin-count waiter */
    int           freeNext;              /* link in freelist chain */
    LWLock        content_lock;          /* to lock access to buffer contents */
} BufferDesc;
```


pg_atomic_uint32 state

```
#define BUF_REFCOUNT_ONE 1
#define BUF_REFCOUNT_MASK ((1U << 18) - 1)
#define BUF_USAGECOUNT_MASK 0x003C0000U
#define BUF_USAGECOUNT_ONE (1U << 18)
#define BUF_USAGECOUNT_SHIFT 18
#define BUF_FLAG_MASK 0xFFC00000U

/* Get refcount and usagecount from buffer state */
#define BUF_STATE_GET_REFCOUNT(state) ((state) & BUF_REFCOUNT_MASK)
#define BUF_STATE_GET_USAGECOUNT(state) (((state) & BUF_USAGECOUNT_MASK) >> BUF_USAGECOUNT_SHIFT)

#define BM_LOCKED (1U << 22) /* buffer header is locked */
#define BM_DIRTY (1U << 23) /* data needs writing */
#define BM_VALID (1U << 24) /* data is valid */
#define BM_TAG_VALID (1U << 25) /* tag is assigned */
#define BM_IO_IN_PROGRESS (1U << 26) /* read or write in progress */
#define BM_IO_ERROR (1U << 27) /* previous I/O failed */
#define BM_JUST_DIRTIED (1U << 28) /* dirtied since write started */
#define BM_PIN_COUNT_WAITER (1U << 29) /* have waiter for sole pin */
#define BM_CHECKPOINT_NEEDED (1U << 30) /* must write for checkpoint */
#define BM_PERMANENT (1U << 31) /* permanent buffer (not unlogged,
or init fork) */
```



Buffer Extensions

pg_buffercache

- Auslesen und überwachen von Shared Memory Buffern
- Liest Buffer Descriptors ein und erstellt lokale Arbeitskopie

```
regression=# SELECT * FROM pg_buffercache_usage_counts();
 usage_count | buffers | dirty | pinned 
-----+-----+-----+-----
          0 |   14650 |      0 |      0 
          1 |    1436 |    671 |      0 
          2 |     102 |     88 |      0 
          3 |      23 |     21 |      0 
          4 |       9 |       7 |      0 
          5 |     164 |    106 |      0 
(6 rows)
```

pg_prewarm

- Prefetching von Relationsdaten
- Lädt einen Bereich von Buffer Pages einer Relation in den Buffercache
- Kann auch automatisiert arbeiten mit 2 Worker Threads
- 3 prewarm Methoden
 - Async prefetch vom OS (prefetch)
 - Sync read vom OS (read)
 - In den Postgres Buffercache laden (buffer)

```
for (i = 0; i < NBuffers; i++)
{
    BufferDesc *bufHdr;
    uint32  buf_state;

    bufHdr = GetBufferDescriptor(i);

    /* Lock each buffer header before inspecting. */
    buf_state = LockBufHdr(bufHdr);

    fctx→record[i].bufferid = BufferDescriptorGetBuffer(bufHdr);
    fctx→record[i].relfilenumber = BufTagGetRelNumber(&bufHdr→tag);
    fctx→record[i].reltablespace = bufHdr→tag.spcOid;
    fctx→record[i].reldatabase = bufHdr→tag.dbOid;
    fctx→record[i].forknum = BufTagGetForkNum(&bufHdr→tag);
    fctx→record[i].blocknum = bufHdr→tag.blockNum;
    fctx→record[i].usagecount = BUF_STATE_GET_USAGECOUNT(buf_state);
    fctx→record[i].pinning_backends = BUF_STATE_GET_REFCOUNT(buf_state);

    ...

    UnlockBufHdr(bufHdr, buf_state);
}
```

```
/* Open relation and check privileges. */
rel = relation_open(rel0id, AccessShareLock);
aclresult = pg_class_aclcheck(rel0id, GetUserId(), ACL_SELECT);
if (aclresult  $\neq$  ACLCHECK_OK)
    aclcheck_error(aclresult, get_relkind_objtype(rel->rd_rel->relkind), get_rel_name(rel0id));

nblocks = RelationGetNumberOfBlocks(rel);

for (block = first_block; block  $\leq$  nblocks; ++block)
{
    Buffer buf;
    buf = ReadBuffer(rel, block);
    ReleaseBuffer(buf);
    ++blocks_done;
}
```

Remove Pages

```
void DropRelationsAllBuffers(struct SMgrRelationData **smgr_reln, int nlocators);  
void DropDatabaseBuffers(Oid dbid);
```

```
uint32 buf_state;  
buf_state = LockBufHdr(bufHdr);  
InvalidateBuffer(bufHdr);  
UnlockBufHdr(bufHdr, buf_state);
```

```
typedef enum ForkNumber  
{  
    InvalidForkNumber = -1,  
    MAIN_FORKNUM = 0,  
    FSM_FORKNUM,  
    VISIBILITYMAP_FORKNUM,  
    INIT_FORKNUM  
  
} ForkNumber;
```



Abschluss

[1] www.postgresql.org

[2] [Postgres Quellcode](#)

[3] notes.dinhhuu258.dev/database/postgres/buffer_manager

[4] cgi.cse.unsw.edu.au/~cs9315/21T1/lectures/pg-buffers/slides.html

schaut in `src/backend/storage/buffer/bufmgr.c` nach!!!